



**UNIVERSITATEA
TEHNICĂ
DIN CLUJ-NAPOCA**

Aplicație casă de producție filme

Proiectare Software

Autor: Stoleru Alexia-Ioana

Grupa: 30235

FACULTATEA DE AUTOMATICA
SI CALCULATOARE

23 Martie 2026

Cuprins

1	Enunțul problemei	2
2	Instrumente utilizate	2
3	Justificarea stivei tehnologice (Tech Stack)	2
4	Arhitectura Sistemului și Context Map	3
4.1	Delimitarea Contextelor (Bounded Contexts)	3
4.2	Shared Kernel (Common Types)	3
5	Design Patterns Utilizate	3
5.1	Faza de analiză	4
5.1.1	Diagrama de Use-Case	4
5.1.2	Diagrama de Activitate	4
5.2	Faza de proiectare și implementare	5
5.2.1	Diagrama de Clase (Domain Model)	5
5.2.2	Harta de Context (Context Map)	6
5.3	Microserviciile: Descriere Arhitecturală	6
5.3.1	Microserviciul Movies	6
5.3.2	Microserviciul Staff	6
5.3.3	Microserviciul Users	6
5.3.4	Microserviciul Reporting	7
6	Testarea și Validarea Endpoint-urilor (Swagger UI)	7
6.1	Scenarii de Testare a Securității (Security Flow)	7
6.1.1	Testul 1: Înregistrarea Utilizatorului (Endpoint: <code>/api/users/register</code>)	7
6.1.2	Testul 2: Autentificarea și Generarea JWT (Endpoint: <code>/api/users/login</code>)	8
6.1.3	Testul 3: Accesarea Resurselor Protejate (Endpoint: <code>/api/users/all</code>)	9

1 Enunțul problemei

Dezvoltați o aplicație soft care poate fi utilizată de către o **casă de producție filme** pentru administrarea filmelor realizate de aceea casă de producție. Softul va permite:

- Adăugarea, ștergerea și actualizarea informațiilor despre actori, regizori, scenariști;
- Vizualizarea listei actorilor, regizorilor și a scenariștilor;
- Vizualizarea listei filmelor sortată după tip, incluzând:
 - redarea unor imagini din film; între 1 și 3 imagini pentru fiecare film;
 - date despre regizorul și scenaristul filmului;
 - lista actorilor care fac parte din distribuția acelui film;
- Filtrarea filmelor după anumite criterii: tip film (artistic sau serial), categorie film, anul realizării;
- Căutarea tuturor filmelor în care joacă un actor selectat.

2 Instrumente utilizate

Pentru dezvoltarea aplicației de management al producției cinematografice, au fost utilizate următoarele instrumente software:

- **IntelliJ IDEA**: Mediul de dezvoltare integrat (IDE) principal, utilizat pentru scrierea codului sursă și gestionarea dependențelor.
- **Spring Boot 3.2.4**: Framework-ul utilizat pentru backend, facilitând inversiunea controlului (IoC) și injecția dependențelor.
- **JavaFX 21**: Setul de instrumente utilizat pentru construcția interfeței grafice (GUI), oferind o experiență de utilizare interactivă.
- **Hibernate / Spring Data JPA**: Utilizat pentru maparea obiectual-relațională (ORM), permițând interacțiunea cu baza de date prin obiecte Java.
- **MySQL**: Sistemul de gestiune a bazelor de date relaționale utilizat pentru stocarea persistentă a datelor.
- **Scene Builder**: Instrument vizual utilizat pentru design-ul fișierelor FXML corespunzătoare interfeței.
- **Maven**: Instrument pentru managementul și automatizarea build-ului proiectului.

3 Justificarea stivei tehnologice (Tech Stack)

Alegerea limbajului **Java** și a framework-urilor asociate pentru acest proiect se bazează pe următoarele argumente tehnice și arhitecturale:

- **Robusteză și Tipizare Puternică (Java)**: Limbajul previne o gamă largă de erori comune prin verificarea riguroasă a tipurilor de date la compilare, fiind standardul industrial pentru sistemele enterprise de tip backend.
- **Independența Microserviciilor (Spring Boot 3)**: A fost ales ca framework principal deoarece permite crearea de aplicații *stand-alone* (având serverul Tomcat încorporat). Funcționalitatea sa de auto-configurare reduce semnificativ codul *boilerplate*, permițând concentrarea pe logica de business din fiecare domeniu.
- **Securitate Distribuită (Spring Security & JWT)**: Pentru controlul accesului, s-a optat pentru o abordare *stateless* (fără sesiune pe server) folosind **JSON Web Tokens (JWT)**. Această decizie este critică într-o arhitectură cu microservicii: odată ce serviciul de identitate (Users) emite token-ul, orice alt serviciu din rețea (ex: Movies) poate valida

rolul utilizatorului (Admin, Manager, Worker) validând exclusiv semnătura criptografică, fără a mai efectua interogări de rețea.

- **Documentare API Automată (OpenAPI / Swagger):** Integrat prin biblioteca `springdoc-openapi`, Swagger a fost esențial pentru standardizarea contractelor REST. Acesta auto-generează documentația interactivă a endpoint-urilor, asigurând că echipa de frontend (JavaFX) cunoaște exact structura JSON-urilor necesare pentru comunicare.

4 Arhitectura Sistemului și Context Map

4.1 Delimitarea Contextelor (Bounded Contexts)

Sistemul este împărțit în următoarele contexte delimitate conform principiilor Domain-Driven Design (DDD):

- **Movies Context:** Gestionează nucleul aplicației (Filme, Categori, Genuri și Galeria de imagini).
- **Staff Context:** Administrează entitățile de tip personal artistic (Actori, Regizori, Scenariști).
- **Users Context:** Se ocupă de autentificare, autorizare JWT și gestionarea ierarhiei de roluri (Admin, Manager, Worker).
- **Notifications Context:** Un serviciu asincron care procesează alertele de modificare a conturilor către utilizatori.
- **Reporting Context:** Context specializat în agregarea datelor inter-servicii pentru statistici și export (CSV, JSON, XML).

4.2 Shared Kernel (Common Types)

Pentru a permite comunicarea între microservicii fără a partaja obiecte de domeniu (*Domain Leakage*), am definit un pachet de tipuri comune ce conține **Value Objects** de tip identificador (ex: `MovieID`, `ActorID`, `UserID`). Acest aspect permite referențierea între contexte exclusiv prin identitate.

5 Design Patterns Utilizate

Conform cerințelor de implementare a principiilor SOLID și a bunei arhitecturi, au fost integrate următoarele șabloane:

- **Strategy Pattern:** Utilizat extensiv în `ReportingService` pentru a schimba algoritmi de parsare și export de date (CSV vs. XML vs. JSON) în mod dinamic, la cererea clientului.
- **Factory Method:** Folosit pentru delegarea instanțierii tipurilor de notificări (`EmailNotification`, `SMSNotification`).
- **Repository Pattern:** Abstractizează accesul la mediul de persistență prin interfețele Spring Data JPA, ascunzând complexitatea interogărilor SQL.
- **Observer Pattern / Event-Driven:** Utilizat pentru a decupla logica de notificare. `UserService` emite un eveniment atunci când un cont este modificat, iar `NotificationService` îl interceptează.
- **DTO (Data Transfer Object):** Model arhitectural utilizat obligatoriu la frontierele aplicației (Controllers) pentru a transporta datele între microservicii și client (JavaFX), protejând starea internă a entităților de domeniu și ascunzând date sensibile precum hash-urile parolelor.

5.1 Faza de analiză

5.1.1 Diagrama de Use-Case

Descriere: Această diagramă identifică actorii sistemului (Angajat, Manager, Administrator) și funcționalitățile principale accesibile fiecăruia. În contextul actual, evidențiază modul în care cerințele specifice (ex: gestiunea personalului artistic sau vizualizarea graficelor statistice) sunt limitate la anumite roluri, stabilind baza pentru matricea de securitate a aplicației.

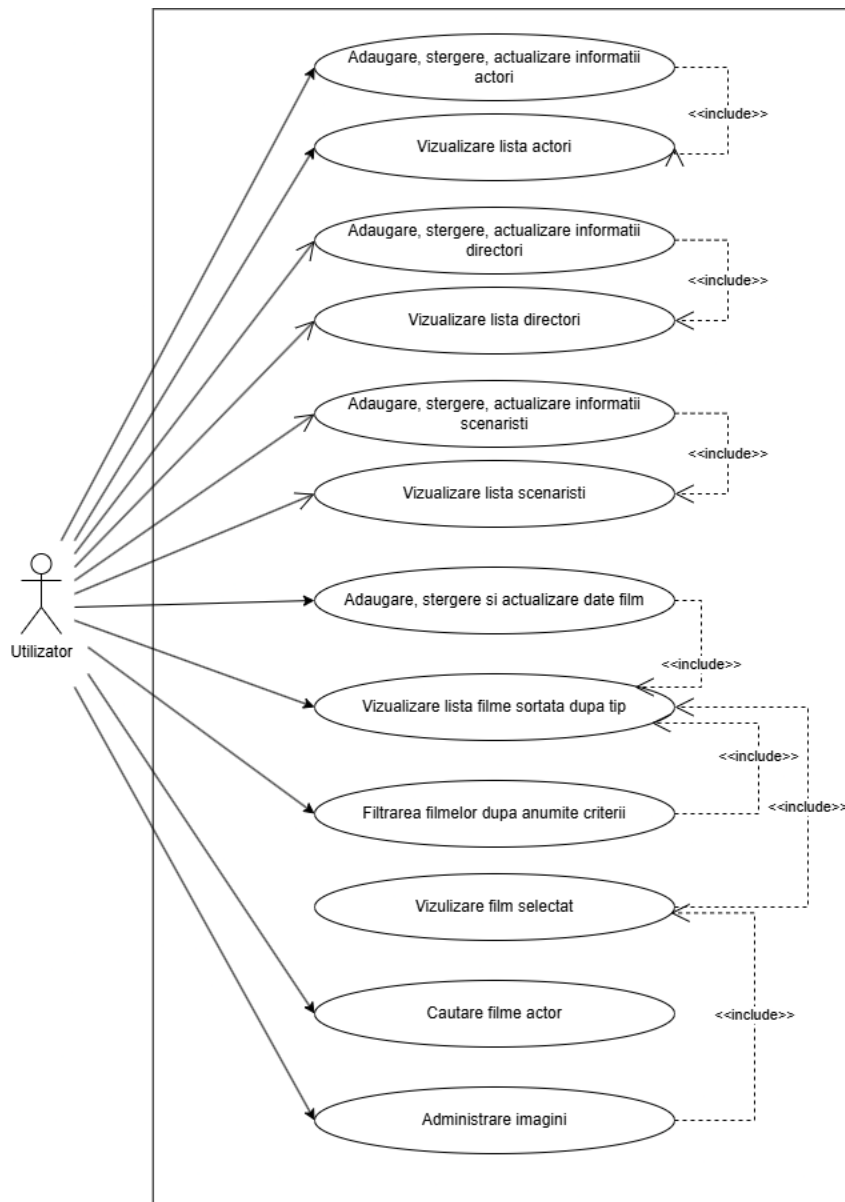


Figura 1: UseCase diagram

5.1.2 Diagrama de Activitate

Descriere: Ilustrează fluxul logic al proceselor complexe inter-domenii, cum ar fi „Adăugarea unui film cu distribuție completă”. Diagrama trasează etapele operaționale, de la validarea primară a metadatelor filmului în *MoviesContext*, până la asocierea cu ID-urile actorilor existenți în *StaffContext*, tratând fluxurile de excepție.

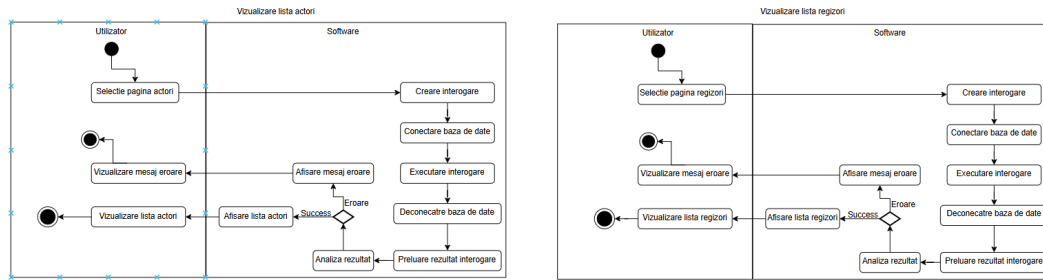


Figura 2: Parte din diagrama de activitate

5.2 Faza de proiectare și implementare

5.2.1 Diagrama de Clase (Domain Model)

Descriere: Surprinde structura statică la nivel de domeniu de business. Abordarea centrală se bazează pe utilizarea obiectelor de tip *Aggregate Root* și evitarea asocierilor directe între entități din microservicii diferite, asigurând coeziunea înaltă a fiecărui modul.

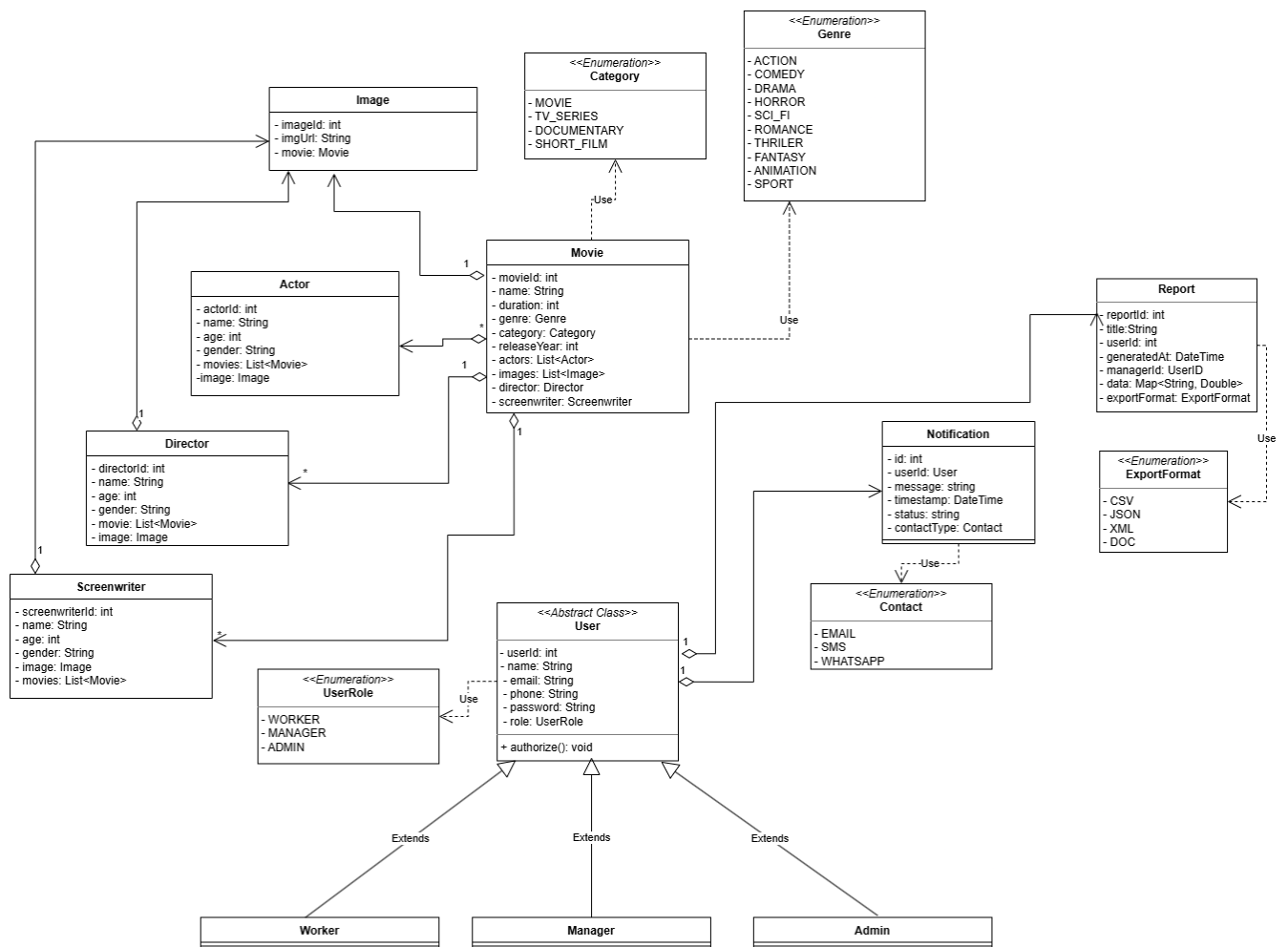


Figura 3: Diagrama de clase (Domeniu)

5.2.2 Harta de Context (Context Map)

Descriere: Reprezintă topologia arhitecturală a celor 5 contexte delimitate. Aceasta documentează relațiile de tip *Upstream/Downstream* și modul în care datele tranzitează granițele microserviciilor prin *Shared Kernel*, demonstrând o arhitectură slab cuplată.

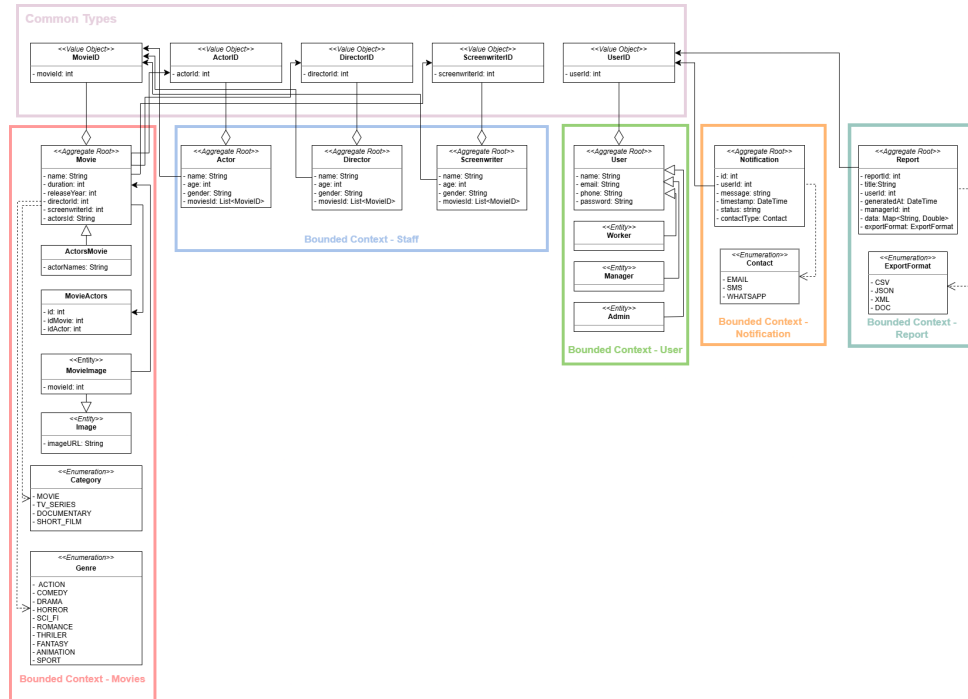


Figura 4: Diagrama de context

5.3 Microserviciile: Descriere Arhitecturală

5.3.1 Microserviciul Movies

Acest serviciu reprezintă inima comercială a casei de producție.

- **Diagrama de Clase:** Definește entitatea `Movie` ca *Aggregate Root*. Prezintă o relație de compoziție cu `Image` (pentru limita de 1-3 imagini promoționale) și liste de *Value Objects* pentru personalul implicat (`DirectorID`, `ActorID`).
- **Diagrama ERD:** Modelează tabelul principal `movies` și tabelele asociative secundare. Asocierile cu distribuția sunt realizate prin tabele de joncțiune care stochează strict chei numerice externe, fără a forța *Foreign Keys* peste granițele bazei de date distribuite.

5.3.2 Microserviciul Staff

Gestionează activele umane artistice.

- **Diagrama de Clase:** Demonstrează aplicarea polimorfismului printr-o structură de bază abstractă din care derivă `Actor`, `Director` și `Screenwriter`.
- **Diagrama ERD:** Utilizează strategii de mapare JPA (*Joined Table* sau *Table per Class*) pentru a stoca eficient atributele specifice fiecărei bresle artistice.

5.3.3 Microserviciul Users

Centralizează mecanismele de autentificare (Authentication) și autorizare (Authorization).

- **Diagrama de Clase:** Ierarhia rolurilor din sistem (**Admin**, **Manager**, **Worker**). Include componentele de infrastructură pentru generarea de **JwtToken** și criptarea parolelor cu **BCrypt**.
- **Diagrama ERD:** Un model relațional simplu bazat pe tabelul **users**, optimizat pentru performanță ridicată la interogările de login.

5.3.4 Microserviciul Reporting

Un serviciu de nivel superior (Downstream) orientat pe consumul și formatarea datelor.

- **Diagrama de Clase:** Demonstrează interacțiunea dintre **ReportGenerator** și multiple implementări ale interfeței **ExportStrategy** (JSON, CSV).
- **Comunicare Inter-servicii:** Utilizează clienți HTTP (ex: **RestTemplate** sau **Feign**) pentru a interoga **MoviesContext** și **UsersContext**, agregând rezultatele pentru rapoartele manageriale și afișarea statisticilor.

6 Testarea și Validarea Endpoint-urilor (Swagger UI)

Pentru validarea arhitecturii REST și a mecanismelor de securitate din **UsersContext**, testarea s-a realizat folosind interfața grafică auto-generată **Swagger (OpenAPI 3)**. Această etapă a confirmat corectitudinea implementării pe mai multe niveluri:

- **Maparea Funcțională:** Interfața a demonstrat respectarea semantică a acțiunilor HTTP: utilizarea metodei **GET** exclusiv pentru obținerea de liste (ex: **/api/users/all**) și a metodei **POST** pentru crearea de resurse (**/api/users/register**).
- **Securitatea Filtrului JWT:** Testele au demonstrat eficiența clasei **JwtAuthenticationFilter**. O cerere neautentificată a returnat corect **403 Forbidden**. Ulterior, prin injectarea unui token valid (generat prin endpoint-ul de Login) în header-ul de tip *Bearer*, filtrul a extras cu succes rolul utilizatorului și a autorizat acțiunea, returnând **200 OK**.
- **Izolarea prin DTO-uri:** În specificațiile vizuale din secțiunea *Schemas*, se observă clar că răspunsurile de succes nu expun entitatea JPA **User** (care ar divulga structura bazei de date și hash-ul de parolă), ci returnează obiecte specifice (**MessageResponseDTO**, **JwtResponseDTO**). Acest comportament dovedește decuplarea straturilor aplicației și asigură securitatea datelor expuse.

6.1 Scenarii de Testare a Securității (Security Flow)

Pentru a demonstra eficiența arhitecturii de securitate *stateless* (bazată pe token-uri), s-a definit un flux de testare în trei pași secvențiali, simulând comportamentul real al unui utilizator.

6.1.1 Testul 1: Înregistrarea Utilizatorului (Endpoint: **/api/users/register**)

Obiectiv: Validarea creării unui cont nou și a mecanismului de *hashing* a parolei.

- **Acțiune:** S-a trimis o cerere **POST** conținând un obiect JSON de tip **RegisterRequestDTO** cu datele utilizatorului (username, parolă în clar, email, telefon și rol).
- **Rezultat Așteptat și Obținut:** Serverul a returnat statusul **201 Created** și un corp de răspuns formatat (**MessageResponseDTO**) cu mesajul de succes.
- **Validarea Bazei de Date:** O verificare în consolă (sau în MySQL) confirmă că entitatea a fost persistată corect. Esențial pentru securitate, coloana corespunzătoare parolei nu stochează șirul de caractere trimis de client, ci hash-ul generat de clasa **BCryptPasswordEncoder**.

(ex: \$2a\$10\$...), demonstrând că baza de date este protejată chiar și în cazul unei breșe de securitate.

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/api/users/register' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "Alexia",
    "email": "alexia.toleru26@gmail.com",
    "phone": "0754799487",
    "password": "parola123",
    "role": "MANAGER"
  }'
```

Request URL

http://localhost:8080/api/users/register

Server response

Code	Details
201	Response body <pre>{ "message": "User registered successfully" }</pre> Response headers <pre>cache-control: no-cache, no-store, max-age=0, must-revalidate connection: keep-alive content-type: application/json date: Tue, 28 Apr 2025 14:00:24 GMT expires: 0 keep-alive: timeout=60 pragma: no-cache transfer-encoding: chunked x-content-type-options: nosniff x-frame-options: DENY x-xss-protection: 0</pre>

Responses

Code	Description	Links
200	OK	No links

Media type

/

Controls Accept header

Example Value

Schema

```
{
  "message": "string"
}
```

Figura 5: Register request

6.1.2 Testul 2: Autentificarea și Generarea JWT (Endpoint: /api/users/login)

Obiectiv: Verificarea procesului de *login* și a generării corecte a token-ului de acces.

- **Acțiune:** S-a efectuat un apel POST trimițând doar **username** și **password** (încapsulate în `LoginRequestDTO`).
- **Rezultat Așteptat și Obținut:** În urma validării cu succes a hash-ului parolei, serverul a returnat statusul 200 OK și un obiect `JwtResponseDTO` care conține șirul de caractere criptografic (JSON Web Token).
- **Observație Arhitecturală:** Acest token conține informații codificate (*claims*), precum ID-ul și rolul utilizatorului, fiind semnat digital cu o cheie secretă cunoscută doar de server.

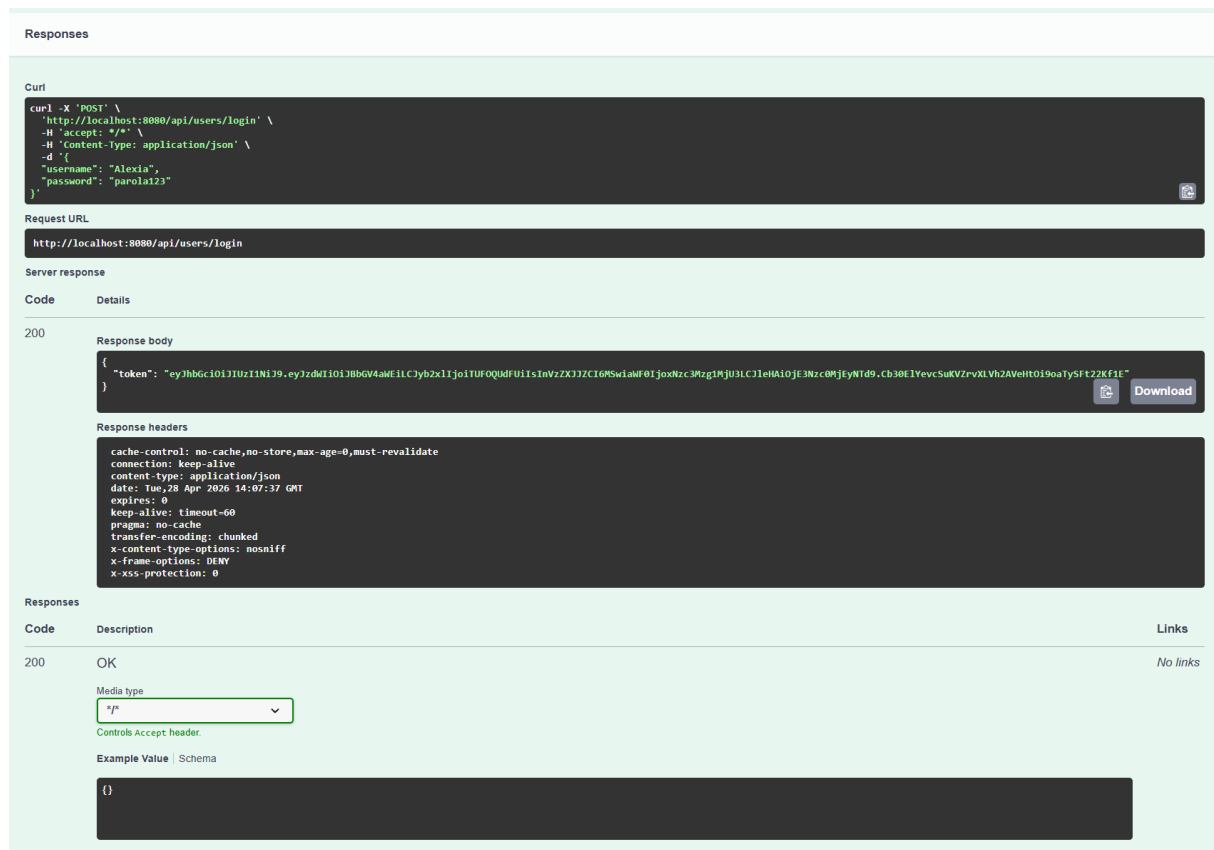


Figura 6: Login request

6.1.3 Testul 3: Accesarea Resurselor Protejate (Endpoint: /api/users/all)

Obiectiv: Validarea filtrului de securitate (`JwtAuthenticationFilter`) și a controlului de acces bazat pe roluri (RBAC). Testul s-a desfășurat în două faze:

1. **Faza de Respingere (Acces Neautorizat):**

- S-a apelat endpoint-ul `GET /api/users/all` *fără* a include token-ul JWT în antetul cererii.
- **Rezultat:** Serverul a interceptat cererea prin filtrul de securitate și a returnat imediat statusul `403 Forbidden`, demonstrând că resursele sunt inaccesibile vizitatorilor anonimi.

2. Faza de Succes (Acces Autorizat):

- Utilizând funcționalitatea ”Authorize” din Swagger, s-a injectat token-ul JWT obținut la Testul 2 în antetul HTTP (sub formatul **Bearer {token}**).
- **Rezultat:** Cererea către același endpoint GET a fost procesată cu succes. Filtrul a validat semnătura token-ului, a extras identitatea și drepturile utilizatorului și a permis executarea codului din **UserController**. Serverul a returnat statusul 200 OK împreună cu lista formatată a utilizatorilor (ca array de obiecte **UserDTO**).

Responses

Curl

```
curl -X 'GET' \
'http://localhost:8080/api/users/all' \
-H 'accept: */*'
```

Request URL

http://localhost:8080/api/users/all

Server response

Code	Details
200	Response body <pre>{ "id": 1, "username": "Alexia", "email": "alexinstolerv26@gmail.com", "phone": "0754799487", "userRole": "MANAGER" }</pre> Download

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-type: application/json
date: Tue, 28 Apr 2026 14:08:48 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 0
```

Responses

Code	Description	Links
200	OK Media type */* Controls Accept header. Example Value Schema <pre>{ "id": 0, "username": "string", "email": "string", "phone": "string", "userRole": "WORKER" }</pre>	

 No links |

Figura 7: Get all users